Linux Commands Study Material

This sheet explains the basic Linux commands you listed.

Online Practice Site

Practice these commands here:

https://killercoda.com/playgrounds/scenario/ubuntu

Course Structure

- Day 1: Basic file, directory, and terminal commands
- Day 2: File management, text processing, and disk usage commands
- Day 3: Package management, networking, processes, compression, and user management

Day 1 Commands

1. `ls`

Purpose: Lists files and folders in the current directory.

Syntax:

```
ls
```

Example:

```
ls
```

Use it when you want to see what is inside your current folder.

2. `ls -a`

Purpose: Lists all files and folders, including hidden ones.

Syntax:

```
ls -a
```

Example:

```
ls -a
```

Note:

- Hidden files in Linux usually begin with a dot, like `.bashrc`.

3. `ls -s`

Purpose: Shows file sizes in blocks along with file and folder names.

Syntax:

```
ls -s
```

Example:

```
ls -s
```

Use it when you want a quick idea of how much space files are using.

4. `ls -R`

Purpose: Lists files and folders recursively, including subdirectories.

Syntax:

```
ls -R
```

Example:

```
ls -R
```

Use it when you want to see the full folder structure.

5. `mkdir dir`

Purpose: Creates a new directory named `dir`.

Syntax:

```
mkdir dir
```

Example:

```
mkdir notes
```

This creates a folder called `notes`.

6. `mkdir dir1 dir2 dir3`

Purpose: Creates multiple directories in one command.

Syntax:

```
mkdir dir1 dir2 dir3
```

Example:

```
mkdir html css js
```

This creates three folders: `html`, `css`, and `js`.

Note:

- You wrote `mkdir dir1 dir2 dir 3`. In Linux, a space separates names, so that would create `dir1`, `dir2`, `dir`, and `3`.
- If you want one folder named `dir 3`, use quotes:

```
mkdir "dir 3"
```

7. `clear`

Purpose: Clears the terminal screen.

Syntax:

```
clear
```

Example:

```
clear
```

It does not delete files or commands. It only clears what you see on the screen.

8. `ll`

Purpose: Usually shows files in long-list format.

Syntax:

```
ll
```

Common output includes:

- file permissions
- owner name
- file size
- modified date
- file name

Important:

- `ll` is not a standard Linux command in every system.
- In many Linux distributions, it is an alias for:

```
ls -l
```

If `ll` does not work, use:

```
ls -l
```

9. `cd dir`

Purpose: Changes the current directory to `dir`.

Syntax:

```
cd dir
```

Example:

```
cd notes
```

After this command, you will move into the `notes` folder.

10. `cd ..`

Purpose: Moves one level up to the parent directory.

Syntax:

```
cd ..
```

Example:

```
cd ..
```

If you are inside `/home/user/docs`, this command moves you to `/home/user`.

11. `touch file_name`

Purpose: Creates a new empty file.

Syntax:

```
touch file_name
```

Example:

```
touch notes.txt
```

If the file already exists, `touch` updates its timestamp.

12. `touch file1 file2`

Purpose: Creates multiple empty files in one command.

Syntax:

```
touch file1 file2
```

Example:

```
touch index.html style.css
```

This creates both files at the same time.

13. `sudo nano file_name`

Purpose: Opens a file in the `nano` text editor with administrator permission.

Syntax:

```
sudo nano file_name
```

Example:

```
sudo nano notes.txt
```

What it does:

- opens the file in the `nano` editor

- if the file does not exist, `nano` can create it
- `sudo` gives permission to edit protected files

Common keys in `nano`:

- `Ctrl + S` saves the file on some systems
- `Ctrl + X` exits the editor

Important:

- In many Linux systems, the usual save command in `nano` is `Ctrl + O`, then press `Enter`.
- `Ctrl + X` is commonly used to exit.

14. `cat file_name`

Purpose: Displays the contents of a file in the terminal.

Syntax:

```
cat file_name
```

Example:

```
cat notes.txt
```

Use it when you want to quickly read a file without opening an editor.

15. `mkdir -p dir1/dir2/dir3`

Purpose: Creates nested directories in one command.

Syntax:

```
mkdir -p dir1/dir2/dir3
```

Example:

```
mkdir -p project/src/components
```

What it does:

- creates `project`
- creates `src` inside `project`
- creates `components` inside `src`

Note:

- `-p` helps create parent directories automatically if they do not already exist.

16. `cd dir1/dir2/dir3`

Purpose: Moves directly into a nested directory path.

Syntax:

```
cd dir1/dir2/dir3
```

Example:

```
cd project/src/components
```

This moves you directly into the `components` folder.

17. `cd ../../..`

Purpose: Moves up three directory levels.

Syntax:

```
cd ../../..
```

Example:

```
cd ../../..
```

If you are inside `/home/user/project/src/components`, this command moves you back to `/home/user`.

18. `cd ~`

Purpose: Moves directly to the home directory.

Syntax:

```
cd ~
```

Example:

```
cd ~
```

The `~` symbol represents the home directory of the current user.

19. `cd`

Purpose: Also moves directly to the home directory.

Syntax:

```
cd
```

Example:

```
cd
```

This is a shortcut way to return to your home directory.

20. `echo hello`

Purpose: Prints text to the terminal.

Syntax:

```
echo hello
```

Example:

```
echo hello
```

Output:

```
hello
```

Use it when you want to display a message on the screen.

21. `echo "welcome" > file_name`

Purpose: Writes text into a file.

Syntax:

```
echo "welcome" > file_name
```

Example:

```
echo "welcome" > notes.txt
```

What it does:

- creates the file if it does not exist
- writes `welcome` into the file
- replaces old content if the file already has something inside it

Important:

- `>` overwrites the file content.

22. `echo "to linux" >> file_name`

Purpose: Adds text to the end of a file.

Syntax:

```
echo "to linux" >> file_name
```

Example:

```
echo "to linux" >> notes.txt
```

What it does:

- adds new text at the end of the file
- keeps the old content unchanged

Important:

- `>>` appends content instead of replacing it.

Day 2 Commands

23. `cat file1 >> file2`

Purpose: Appends the contents of `file1` to `file2`.

Syntax:

```
cat file1 >> file2
```

Example:

```
cat notes1.txt >> notes2.txt
```

Important:

- `>>` adds content to the end of `file2`.
- existing content in `file2` remains safe.

24. `cat file1 > file2`

Purpose: Copies the contents of `file1` into `file2`.

Syntax:

```
cat file1 > file2
```

Example:

```
cat notes1.txt > notes2.txt
```

Important:

- `>` overwrites the old content of `file2`.
- if `file2` already has data, that data will be replaced.

25. `pwd`

Purpose: Shows the present working directory.

Syntax:

```
pwd
```

Example:

```
pwd
```

Use it when you want to know your current location in the terminal.

26. `rm file_name`

Purpose: Removes a file.

Syntax:

```
rm file_name
```

Example:

```
rm notes.txt
```

Important:

- this permanently deletes the file from the current location.

27. `rm dir`

Purpose: Tries to remove a directory.

Syntax:

```
rm dir
```

Important:

- this usually does not remove a normal directory by itself.
- to remove directories, you often need `rm -r dir`.
- for empty directories, another common command is `rmdir dir`.

28. `rm -r`

Purpose: Removes directories and their contents recursively.

Syntax:

```
rm -r dir_name
```

Example:

```
rm -r project
```

Important:

- this deletes the directory and everything inside it.
- use it very carefully.

29. `cp file_name1 file_name2`

Purpose: Copies the content of one file into another file.

Syntax:

```
cp file_name1 file_name2
```

Example:

```
cp notes.txt backup.txt
```

If `backup.txt` exists, its content will be replaced.

30. `cp file_name dir`

Purpose: Copies a file from the current directory into another directory.

Syntax:

```
cp file_name dir
```

Example:

```
cp notes.txt docs
```

This copies `notes.txt` into the `docs` folder.

31. `cp -r dir1 dir2`

Purpose: Copies one directory and its contents into another location.

Syntax:

```
cp -r dir1 dir2
```

Example:

```
cp -r project backup_project
```

The `-r` option means recursive copy.

32. `mv file_name dir`

Purpose: Moves a file into another directory.

Syntax:

```
mv file_name dir
```

Example:

```
mv notes.txt docs
```

This moves `notes.txt` into the `docs` folder.

33. `mv file_name1 file_name2`

Purpose: Renames a file or moves it with a new name.

Syntax:

```
mv file_name1 file_name2
```

Example:

```
mv notes.txt notes-old.txt
```

This renames `notes.txt` to `notes-old.txt`.

34. `tree`

Purpose: Shows files and folders in a tree-style structure.

Syntax:

```
tree
```

Install first time if not available:

```
sudo apt install tree
```

Use it when you want to see folder structure clearly.

35. `head file_name`

Purpose: Shows the first 10 lines of a file.

Syntax:

```
head file_name
```

Example:

```
head notes.txt
```

36. `head -n 3 file_name`

Purpose: Shows the first 3 lines of a file.

Syntax:

```
head -n 3 file_name
```

Example:

```
head -n 3 notes.txt
```

37. `tail file_name`

Purpose: Shows the last 10 lines of a file.

Syntax:

```
tail file_name
```

Example:

```
tail notes.txt
```

38. `tail -n 3 file_name`

Purpose: Shows the last 3 lines of a file.

Syntax:

```
tail -n 3 file_name
```

Example:

```
tail -n 3 notes.txt
```

39. `stat file_name/dir`

Purpose: Shows detailed information about a file or directory.

Syntax:

```
stat file_name
stat dir_name
```

Example:

```
stat notes.txt
```

It can show size, permissions, owner, and timestamps.

40. `grep`

Meaning: `grep` is commonly explained as Global Regular Expression Print.

Purpose: Searches for matching text inside a file.

41. `grep text file_name`

Purpose: Finds matching text in a file.

Syntax:

```
grep text file_name
```

Example:

```
grep linux notes.txt
```

42. `grep -i text file_name`

Purpose: Searches text without case sensitivity.

Syntax:

```
grep -i text file_name
```

Example:

```
grep -i linux notes.txt
```

This matches `linux`, `Linux`, and `LINUX`.

43. `grep -n text file_name`

Purpose: Searches text and also shows line numbers.

Syntax:

```
grep -n text file_name
```

Example:

```
grep -n linux notes.txt
```

44. `sed 's/old_word/new_word/' file_name`

Purpose: Replaces the first matching word in each line when showing output.

Syntax:

```
sed 's/old_word/new_word/' file_name
```

Example:

```
sed 's/linux/Linux/' notes.txt
```

Important:

- this shows the changed output in the terminal.
- it does not change the original file.

45. `sed -i 's/old_word/new_word/' file_name`

Purpose: Replaces matching text directly inside the file.

Syntax:

```
sed -i 's/old_word/new_word/' file_name
```

Example:

```
sed -i 's/linux/Linux/' notes.txt
```

Important:

- `-i` means in-place editing.
- this changes the real file.

46. `sed '3d' file_name`

Purpose: Deletes line 3 in the displayed output.

Syntax:

```
sed '3d' file_name
```

Example:

```
sed '3d' notes.txt
```

Important:

- this does not change the original file.

47. `sed -i '3d' file_name`

Purpose: Deletes line 3 directly from the file.

Syntax:

```
sed -i '3d' file_name
```

Example:

```
sed -i '3d' notes.txt
```

48. `df`

Purpose: Shows disk space usage.

Syntax:

```
df
```

Example:

```
df
```

49. `df -h`

Purpose: Shows disk space usage in human-readable format.

Syntax:

```
df -h
```

Example:

```
df -h
```

The `-h` option shows sizes like KB, MB, and GB.

50. `sudo su`

Purpose: Switches to the superuser account.

Syntax:

```
sudo su
```

Important:

- this gives administrator-level access.
- use it carefully because you can change system files.

51. `exit`

Purpose: Exits the current shell or user session.

Syntax:

```
exit
```

Example:

```
exit
```

Use it to leave `sudo su`, a terminal session, or a shell.

Day 3 Commands

52. `curl`

Meaning: `curl` stands for Client URL.

Purpose: Transfers data from or to a server.

Syntax:

```
curl URL
```

Example:

```
curl https://example.com
```

Use it to test URLs, APIs, and downloads.

53. `sudo apt update`

Purpose: Updates the package list from configured repositories.

Syntax:

```
sudo apt update
```

Use it before installing or upgrading packages.

54. `sudo apt upgrade`

Purpose: Upgrades installed packages to newer versions.

Syntax:

```
sudo apt upgrade
```

This uses the latest package information downloaded by `apt update`.

55. `sudo apt install package_name`

Purpose: Installs a package.

Syntax:

```
sudo apt install package_name
```

Example:

```
sudo apt install nginx
```

56. `wget`

Meaning: `wget` stands for World Wide Web Get.

Purpose: Downloads files from the web.

Syntax:

```
wget URL
```

Example:

```
wget https://example.com/file.zip
```

57. `ss`

Purpose: Shows socket and network connection information.

Syntax:

```
ss
```

Use it to inspect active network connections.

58. `ss -u`

Purpose: Shows UDP sockets.

Syntax:

```
ss -u
```

59. `ss -t`

Purpose: Shows TCP sockets.

Syntax:

```
ss -t
```

60. `ss -l`

Purpose: Shows listening sockets.

Syntax:

```
ss -l
```

61. `ss -n`

Purpose: Shows numeric addresses and ports without name resolution.

Syntax:

```
ss -n
```

62. `ifconfig`

Purpose: Displays or configures network interfaces.

Syntax:

```
ifconfig
```

Note:

- on some modern Linux systems, `ifconfig` may not be installed by default.

63. `ping www.google.com`

Purpose: Checks network connectivity to a host.

Syntax:

```
ping www.google.com
```

Use `Ctrl + C` to stop the continuous ping output.

64. `which package_name`

Purpose: Shows the path of a command.

Syntax:

```
which package_name
```

Example:

```
which nginx
```

65. `sudo service package_name status`

Purpose: Checks the status of a service.

Syntax:

```
sudo service package_name status
```

Example:

```
sudo service nginx status
```

66. `sudo systemctl status nginx`

Purpose: Shows the current status of the `nginx` service.

Syntax:

```
sudo systemctl status nginx
```

67. `sudo systemctl start nginx`

Purpose: Starts the `nginx` service.

Syntax:

```
sudo systemctl start nginx
```

68. `sudo systemctl stop nginx`

Purpose: Stops the `nginx` service.

Syntax:

```
sudo systemctl stop nginx
```

69. `ps`

Purpose: Shows processes running in the current shell session.

Syntax:

```
ps
```

70. `ps aux`

Purpose: Shows detailed information about all running processes.

Syntax:

```
ps aux
```

71. `top`

Purpose: Shows real-time process activity.

Syntax:

```
top
```

Use `q` to quit.

72. `htop`

Purpose: Shows an interactive process viewer.

Syntax:

```
htop
```

Note:

- `htop` may need to be installed first.

73. `kill pid`

Purpose: Stops a process using its process ID.

Syntax:

```
kill pid
```

Example:

```
kill 1234
```

74. `uname`

Purpose: Shows basic system information.

Syntax:

```
uname
```

75. `uname -a`

Purpose: Shows all available system information.

Syntax:

```
uname -a
```

76. `whoami`

Purpose: Shows the current logged-in username.

Syntax:

```
whoami
```

77. `hostname`

Purpose: Shows the system hostname.

Syntax:

```
hostname
```

78. `sudo hostnamectl set-hostname <name>`

Purpose: Changes the system hostname.

Syntax:

```
sudo hostnamectl set-hostname <name>
```

Example:

```
sudo hostnamectl set-hostname dev-server
```

Note:

- you wrote `audo`, which appears to be a typo. The correct command is `sudo`.

79. `gzip file_name`

Purpose: Compresses a file into gzip format.

Syntax:

```
gzip file_name
```

Example:

```
gzip notes.txt
```

80. `gunzip file_name`

Purpose: Extracts a gzip-compressed file.

Syntax:

```
gunzip file_name.gz
```

Example:

```
gunzip notes.txt.gz
```

81. `unzip file_name`

Purpose: Extracts files from a zip archive.

Syntax:

```
unzip file_name
```

Example:

```
unzip files.zip
```

82. `zip`

Purpose: Creates a zip archive.

Syntax:

```
zip archive_name files
```

Example:

```
zip files.zip notes.txt
```

83. `zcat`

Purpose: Displays the contents of a gzip-compressed file without extracting it manually.

Syntax:

```
zcat file_name.gz
```

Example:

```
zcat notes.txt.gz
```

84. `sudo adduser user_name`

Purpose: Creates a new user account.

Syntax:

```
sudo adduser user_name
```

Example:

```
sudo adduser devuser
```

85. `sudo groupadd group_name`

Purpose: Creates a new group.

Syntax:

```
sudo groupadd group_name
```

Example:

```
sudo groupadd developers
```

86. `id user_name`

Purpose: Shows user ID, group ID, and related group information.

Syntax:

```
id user_name
```

Example:

```
id devuser
```

87. `getent group group_name`

Purpose: Shows information about a specific group.

Syntax:

```
getent group group_name
```

Example:

```
getent group developers
```

88. `sudo usermod -aG group_name user_name`

Purpose: Adds a user to an existing group.

Syntax:

```
sudo usermod -aG group_name user_name
```

Example:

```
sudo usermod -aG developers devuser
```

Important:

- `-aG` appends the user to the group without removing existing group memberships.

89. `groups user_name`

Purpose: Shows the groups a user belongs to.

Syntax:

```
groups user_name
```

Example:

```
groups devuser
```

Note:

- you wrote `group username`, but the common Linux command is `groups user_name`.

90. `cat /etc/passwd`

Purpose: Shows the list of user account entries.

Syntax:

```
cat /etc/passwd
```

91. `getent group`

Purpose: Shows all group entries available on the system.

Syntax:

```
getent group
```

92. `sudo deluser username`

Purpose: Removes a user account.

Syntax:

```
sudo deluser username
```

Example:

```
sudo deluser devuser
```

93. `sudo groupdel groupname`

Purpose: Deletes a group.

Syntax:

```
sudo groupdel groupname
```

Example:

```
sudo groupdel developers
```

Day 1 Quick Revision

Command	Use
`ls`	List files and folders
`ls -a`	List all files, including hidden files
`ls -s`	Show file sizes
`ls -R`	List files recursively
`mkdir dir`	Create one directory
`mkdir dir1 dir2 dir3`	Create multiple directories
`clear`	Clear terminal screen
`ll`	Long list view, often alias of `ls -l`
`cd dir`	Move into a directory
`cd ..`	Move to parent directory
`touch file_name`	Create one empty file
`touch file1 file2`	Create multiple empty files
`sudo nano file_name`	Open or edit a file in nano with sudo
`cat file_name`	Show file contents in terminal
`mkdir -p dir1/dir2/dir3`	Create nested directories
`cd dir1/dir2/dir3`	Move into a nested directory path
`cd ../../..`	Move up three directory levels
`cd ~`	Move to home directory
`cd`	Move to home directory
`echo hello`	Print text to the terminal
`echo "welcome" > file_name`	Write text to a file

Command	Use
`echo "to linux" >> file_name`	Append text to a file

Day 1 Practice

```
mkdir linux_practice
cd linux_practice
touch file1.txt file2.txt
mkdir dir1 dir2 dir3
mkdir -p project/src/components
cd project/src/components
cd ../../..
sudo nano file1.txt
echo "welcome" > file1.txt
echo "to linux" >> file1.txt
cat file1.txt
echo hello
ls
ls -a
ls -s
ls -R
cd dir1
cd ..
cd ~
cd
clear
```

Day 2 Quick Revision

Command	Use
`cat file1 >> file2`	Append file1 content into file2
`cat file1 > file2`	Copy file1 content into file2
`pwd`	Show present working directory
`rm file_name`	Remove a file
`rm dir`	Try to remove a directory
`rm -r dir_name`	Remove a directory recursively

Command	Use
`cp file_name1 file_name2`	Copy one file into another
`cp file_name dir`	Copy a file into a directory
`cp -r dir1 dir2`	Copy a directory recursively
`mv file_name dir`	Move a file into a directory
`mv file_name1 file_name2`	Rename or move a file
`tree`	Show directory structure in tree form
`head file_name`	Show first 10 lines
`head -n 3 file_name`	Show first 3 lines
`tail file_name`	Show last 10 lines
`tail -n 3 file_name`	Show last 3 lines
`stat file_name/dir`	Show detailed file or directory info
`grep text file_name`	Search text in a file
`grep -i text file_name`	Search text ignoring case
`grep -n text file_name`	Search text with line numbers
`sed 's/old_word/new_word/' file_name`	Replace text in output only
`sed -i 's/old_word/new_word/' file_name`	Replace text inside the file
`sed '3d' file_name`	Delete line 3 in output only
`sed -i '3d' file_name`	Delete line 3 from the file
`df`	Show disk space usage
`df -h`	Show human-readable disk space
`sudo su`	Switch to superuser
`exit`	Exit the current shell

Day 2 Practice

```
pwd
cat file1.txt > copy.txt
cat file1.txt >> copy.txt
head file1.txt
head -n 3 file1.txt
tail file1.txt
tail -n 3 file1.txt
grep welcome file1.txt
grep -i welcome file1.txt
grep -n welcome file1.txt
sed 's/welcome/Welcome/' file1.txt
stat file1.txt
cp file1.txt backup.txt
mv backup.txt dir1
tree
df
df -h
sudo su
exit
```

Day 3 Quick Revision

Command	Use
`curl URL`	Transfer data from or to a server
`sudo apt update`	Update package list
`sudo apt upgrade`	Upgrade installed packages
`sudo apt install package_name`	Install a package
`wget URL`	Download a file from the web
`ss`	Show socket information
`ss -u`	Show UDP sockets
`ss -t`	Show TCP sockets
`ss -l`	Show listening sockets

Command	Use
`ss -n`	Show numeric sockets and ports
`ifconfig`	Show network interface information
`ping www.google.com`	Check network connectivity
`which package_name`	Show command path
`sudo service package_name status`	Check service status
`sudo systemctl status nginx`	Show nginx status
`sudo systemctl start nginx`	Start nginx
`sudo systemctl stop nginx`	Stop nginx
`ps`	Show current-shell processes
`ps aux`	Show all running processes
`top`	Real-time process viewer
`htop`	Interactive process viewer
`kill pid`	Stop a process by PID
`uname`	Show basic system info
`uname -a`	Show detailed system info
`whoami`	Show current username
`hostname`	Show system hostname
`sudo hostnamectl set-hostname <name>`	Change system hostname
`gzip file_name`	Compress a file
`gunzip file_name.gz`	Extract a gzip file
`unzip file_name`	Extract a zip file
`zip archive_name files`	Create a zip archive
`zcat file_name.gz`	View gzip content without extracting
`sudo adduser user_name`	Create a user

Command	Use
`sudo groupadd group_name`	Create a group
`id user_name`	Show user and group IDs
`getent group group_name`	Show one group entry
`sudo usermod -aG group_name user_name`	Add user to a group
`groups user_name`	Show user groups
`cat /etc/passwd`	Show user account entries
`getent group`	Show all group entries
`sudo deluser username`	Delete a user
`sudo groupdel groupname`	Delete a group

Day 3 Practice

```
curl https://example.com
wget https://example.com/file.zip
ss
ss -t
ss -u
ss -l
ss -n
ifconfig
ping www.google.com
which nginx
sudo service nginx status
sudo systemctl status nginx
ps
ps aux
uname
uname -a
whoami
hostname
gzip copy.txt
gunzip copy.txt.gz
zip files.zip file1.txt
unzip files.zip
sudo adduser devuser
sudo groupadd developers
```

```
id devuser
getent group developers
groups devuser
cat /etc/passwd
getent group
```

Key Points to Remember

- Day 1 focuses on navigation, files, folders, and basic terminal usage.
- Day 2 focuses on copying, moving, deleting, searching, and editing text.
- Day 3 focuses on packages, networking, processes, compression, and user management.
- `cd ~` and `cd` both take you to the home directory.
- `rm -r` removes folders and their contents, so use it carefully.
- `grep` searches text inside files, while `sed` changes or transforms text.
- `apt` is used to update, upgrade, and install software.
- `curl` and `wget` are helpful for testing URLs and downloading files.
- `ps`, `top`, and `htop` help monitor processes.
- `gzip`, `gunzip`, `zip`, and `unzip` handle file compression.
- `adduser`, `groupadd`, and `usermod` are used for user and group management.
- `ll` may not work on every Linux system.